

Slide Design Rules - Read this carefully

1. Lesson slides must use consistent typography across all pages.
2. Text must remain readable on mobile, tablet and desktop.
3. Use responsive design principles, not whole-slide shrinking.
4. Use shared CSS design tokens for typography, spacing and layout.
5. Do not use random one-off font sizes inside individual slides.
6. Avoid tiny text such as 8px, 9px, 10px or 11px.
7. Avoid `transform: scale()` to shrink whole slides.
8. Avoid viewport-only font sizing that becomes unreadable.
9. Use `clamp()` only with safe minimum sizes.
10. Desktop, tablet and mobile must each have minimum text sizes.
11. Mobile layouts must stack vertically where needed.
12. Diagrams, tables and interactions must remain readable on smaller screens.
13. Fixed bottom navigation must not cover slide content.
14. Slides should behave like responsive webpages, not fixed PowerPoint slides.
15. Main slide titles must always span the whole slide width.
16. Main slide titles must sit above the content grid, not inside one column.
17. A slide must not require scrolling to view core content.
18. If content does not fit, Codex must remove, split or simplify redundant content rather than shrinking the slide or cutting content off.
19. Slides must maintain balanced visual weight across the canvas.
20. Avoid large unused dead zones unless deliberately used for focus.
21. Two-column layouts must have balanced content density and visual weight.
22. If one side has less content, Codex must resize columns, expand the weaker area, increase visual importance, stack content differently, split content, or redesign the layout.
23. Tables must not appear visually tiny compared with surrounding whitespace.
24. When a table is the primary teaching element, it must dominate the layout and act as the focal point.
25. Use the hinge point slide layout as a positive reference for balance.
26. Codex must select the most suitable approved wireframe/layout archetype based on the teaching intention, rather than forcing every slide into the same reusable component structure.

Learning objectives

By the end of this lesson, you should be able to:

- 1 **Describe** the purpose of the Central Processing Unit.
- 2 **Identify** the ALU, Control Unit, cache and registers.
- 3 **Explain** what the MAR, MDR, Program Counter and Accumulator store.
- 4 **Model** the stages of the fetch-execute cycle.
- 5 **Distinguish** between data and an address.

How does the CPU know what to do next?



Clickable
Question
box

You open a game. The screen changes, sounds play, buttons respond and the score updates. It feels instant, but inside the computer the CPU is repeatedly fetching, decoding and executing instructions.

Question

What does the CPU need before it can carry out an instruction?

LESSON TITLE
1.1.1 Architecture of the CPU

LESSON NAVIGATION

- 1 How does the CPU know what to do next?
- 2 Learning objectives
- 3 CPU and memory work together
- 4 The CPU processes instructions and data

LESSON TOOLS +

TEACHER TOOLS +

• LESSON INTRODUCTION

How does the CPU know what to do next?

• ANSWER

What does the CPU need before it can carry out an instruction?

The CPU needs the instruction, its location in memory, any required data, a way to control the order of events and temporary storage for values being processed.

Close

Question

What does the CPU need before it can carry out an instruction?

Back to topics

Back

Slide 1 of 18: How does the CPU know what to do next?

Next

Light box
Answer
box

The CPU processes instructions and data

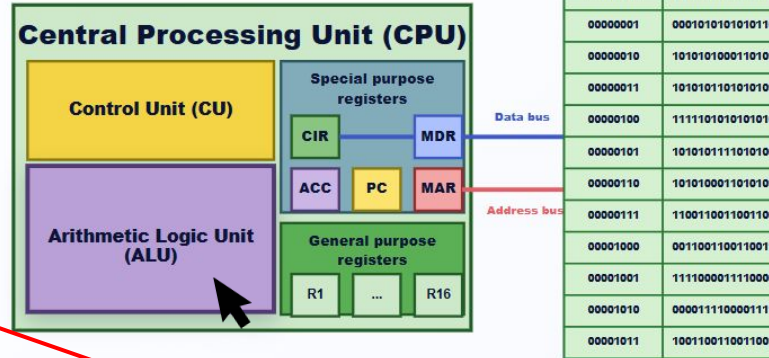
RAM

Address	Instructions / data
1040	Instructions: Spotify play button instructions
1041	Data: Spotify current song data
1042	Instructions: Chrome tab instructions
1043	Data: Chrome webpage image data
1044	Data: Game score data

A program is made from instructions and data. The instructions and data are stored in RAM. The Central Processing Unit fetches the instructions and data from RAM and executes it.

Explain
Box

Main CPU components



Call to action

Explain Box

Click a component to see what it does.

ALU

The Arithmetic Logic Unit performs calculations and logical comparisons, such as adding numbers or checking whether one value is larger than another.

Check the CPU's purpose

Q1

1 mark

Hinge

What is the main purpose of the CPU?

- A** To permanently store files and programs.
- B** To process instructions and control many computer operations.
- C** To display images on the monitor.
- D** To connect the computer to the Internet.

Correct

WHY

Instructions are stored in memory before the CPU processes them through the fetch-execute cycle.

Answer
Box with
reason

Back

Slide 5 of 18: Check the CPU's purpose

Next

Explain CPU architecture in context

Q3 A calculator program adds two numbers. Explain how the ALU, Accumulator and Control Unit are involved.

4 marks

1 Name the component that performs the addition.

2 Name the register that can store the result.

3 Explain how the Control Unit coordinates the process.

4 Use the scenario, not just definitions.

Mark point 2

WHY

The Accumulator can store the result of the calculation temporarily while the CPU is processing.

2. Main title

3. Image / diagram / animation / table

4. Text explaining the
above

5. Text explaining the
above

1. Modelling

2. Main title

3. Modelling

4. Text explaining the step

Previous step

Next step

1. Interactive table

2. Main title

Label 1	Label 2
Label 3	Label 4
Label 5	Label 6 
Label 7	Label 8

4. Text explaining Label 6

2. Put the stages in the correct order

First:  

Second:  

Third:  

Fourth:  



4.Incorrect

1. Assessment

2. Exam Question

3. Question (x marks)

4. Mark scheme

Reveal Mark scheme

